Software Architecture and Design Specification

Project	IT Infrastructure & Asset Management System
Version	1.0
Authors	SRI RAM V, Sharvesh, Yashwanth, Shreyan Ganesh
Date	17-10-2026
Status	Draft for academic review

Revision History

Version	Date	Author	Change Summary
1.0	17-10-2026	SRI RAM V, Sharvesh, Yashwanth, Shreyan Ganesh	Initial architecture and design specification

Approvals

Role	Name	Signature / Date
Project Guide	To be assigned	
Team Representative	SRI RAM V	

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design of ITAMS, including its components, data responsibilities, security controls, API contracts, key workflows and operational considerations.

1.2 Scope

The design covers inventory, asset lifecycle, allocation, software licenses, warranties, maintenance, alerts, reporting, user/role administration and audit history. Integrations beyond the baseline are optional.

1.3 Audience

Developers, testers, IT administrators, asset managers, project guide and maintainers.

1.4 Definitions

ITAMS: IT Infrastructure & Asset Management System; RBAC: Role-Based Access Control; API: Application Programming Interface; ORM: Object-Relational Mapping; RTM: Requirements Traceability Matrix; ADR: Architecture Decision Record.

2. Document Overview

2.1 How to Use This Document

Use the architecture section to understand module boundaries and deployment. Use the design section for API contracts and workflows. Use the SRS for normative requirements and the test plan for verification procedures.

2.2 Related Documents

ITAMS SRS v1.0; ITAMS Software Test Plan v1.0; project RTM; applicable organizational information-security and asset-management policies.

3. Architecture

3.1 Goals and Constraints

- Centralized and consistent inventory with traceable asset lifecycle events.
- Role-appropriate access and auditability for sensitive changes.
- Reliable date-driven alerts for license/warranty expiry and maintenance.
- Maintainable modular implementation suitable for an academic project and future extension.
- Constraints: data quality depends on supplied records; notification delivery depends on configured service; baseline does not directly discover or control endpoint devices.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concern
Asset Manager / IT Admin	Accurate records, fast search, allocation and expiry alerts
Department Manager	Visibility into departmental assets and responsibility
Employee / Asset Holder	Clear assigned-asset details and return/fault process
Auditor	Traceable changes, compliance reporting and read-only access
Developer / Maintainer	Modularity, testability, secure APIs and maintainability

3.3 Component Architecture Diagram

Logical architecture (baseline):

```
[Browser UI]
    | HTTPS / authenticated session
    v
[Web Controller / API Layer]
    |-- Authentication & RBAC
    |-- Asset Inventory & Lifecycle
    |-- Allocation / Return
    |-- License & Warranty
    |-- Maintenance
    |-- Alert Scheduler / Notification
    |-- Dashboard / Reporting
    |-- Audit Service
    v
[Service / Validation Layer]
    v
[Relational Database]
    |-- Users / Roles
    |-- Assets / Categories / Locations
    |-- Allocations / History
    |-- Licenses / Warranties
    |-- Maintenance / Alerts
    |-- Audit Events
Optional: [SMTP Email Service] <--- Alert Scheduler
```

Component interaction is layered: controllers validate request shape and identity, services enforce business rules, repositories persist data, and the notification adapter sends alerts. Database access is never exposed directly to the browser.

3.4 Component Descriptions

Component	Responsibilities
Web UI	Dashboard, forms, tables, filters, alert centre and reports.
Authentication & RBAC	Login/session validation, password verification, role and permission checks.
Asset Inventory Service	Create/update/search/retire asset records; enforce unique IDs and required fields.
Allocation Service	Assign, return and transfer assets; maintain current holder and history.
License & Warranty Service	Store entitlements, expiry dates, warranty details and renewal status.
Maintenance Service	Schedule work, record repair history, update due dates and status.
Alert Scheduler	Evaluate expiry/overdue rules, avoid duplicate alerts and record delivery/acknowledgement status.
Reporting Service	Aggregate inventory, allocation, maintenance and compliance data; export permitted reports.
Audit Service	Record actor, timestamp, action and affected record for significant changes.
Repository / Database	Persist relational entities and enforce referential integrity.

3.5 Architecture Pattern and Rationale

A layered modular monolith is recommended for the initial implementation. It separates UI, controllers, business services and persistence while keeping deployment and debugging manageable for a small team. Microservices are not required for the current scope because they add deployment, observability and distributed-transaction complexity without a demonstrated scale need. Modules can be separated later if operational demand justifies it.

3.6 Technology Stack and Data Stores

- Frontend: HTML/CSS/JavaScript or React, depending on team implementation.
- Backend: Python Flask/Django or Java Spring Boot; choose one framework consistently for the implementation.
- Database: PostgreSQL or MySQL relational database.
- API payloads: JSON over HTTPS; reports: CSV/PDF where supported.
- Version control and collaboration: Git; API testing: Postman; diagrams: PlantUML/draw.io; automated tests: framework-native test runner.

The above is a proposed stack, not a claim about an already implemented system. The team should select one backend and database before coding and update this document if the actual stack differs.

3.7 Risks and Mitigations

Risk	Mitigation
Duplicate or inaccurate imported data	Validate CSV schema, detect duplicate tags/serials and provide row-level import errors.
Expiry or maintenance alert missed	Run scheduled checks, persist alert state, log failures and provide an alert dashboard fallback.
Unauthorized record changes	Enforce server-side RBAC on every endpoint and test negative permissions.
Sensitive license details exposed	Store only required secrets, mask values, restrict access and exclude secrets from logs.
Database loss or corruption	Schedule backups and periodically test restoration.
Email service unavailable	Retry with bounded backoff, retain in-app alerts and log delivery status.

3.8 Traceability to Requirements

SRS Requirement	Design Component

ITAM-F-001, ITAM-F-002; ITAM-SR-002	Authentication & RBAC
ITAM-F-003 to ITAM-F-006	Asset Inventory Service
ITAM-F-007, ITAM-F-008	Allocation Service
ITAM-F-009 to ITAM-F-011	License & Warranty Service
ITAM-F-012, ITAM-F-013	Maintenance Service
ITAM-F-014 to ITAM-F-016	Alert Scheduler / Alert Centre
ITAM-F-017 to ITAM-F-019	Dashboard / Reporting Service
ITAM-F-020	Audit Service
ITAM-F-021, ITAM-F-022	Lifecycle / Import Service

3.9 Security Architecture

- Authentication: use secure password hashing, session expiry and login throttling where supported.
- Authorization: deny by default; enforce RBAC in backend services, not only by hiding UI controls.
- Transport: HTTPS/TLS in deployed environments; secure cookies and CSRF protection where session-based web forms are used.
- Data protection: validate input, parameterize database queries, protect backups and avoid storing unnecessary personal data.
- Audit: record actor, timestamp, action and target record; do not log passwords, tokens or full license secrets.
- STRIDE considerations: spoofing—authentication; tampering—authorization and audit; repudiation—append-oriented audit trail; information disclosure—least privilege and encryption; denial of service—rate limits and bounded uploads; elevation of privilege—server-side RBAC and secure configuration.

4. Design

4.1 Design Overview

The design uses normalized relational entities and service-level business rules. Asset status and allocation state should be updated within a transaction so the system cannot allocate an already-retired or unavailable asset. Date-driven alert generation should be idempotent to avoid repeated duplicate alerts for the same asset, alert type and expiry window.

4.2 UML Sequence Diagrams

Flow A: Allocate an Asset

```
Asset Manager -> Web UI: Select asset and assignee
Web UI -> API: POST /api/allocations
API -> Auth/RBAC: Verify session and allocation permission
Auth/RBAC --> API: Authorized
API -> Allocation Service: validate availability and assignee
Allocation Service -> Database: Begin transaction; read asset
Database --> Allocation Service: Asset status/current assignment
Allocation Service -> Database: Create allocation + update asset status + audit event
Database --> Allocation Service: Commit
Allocation Service --> API: Allocation ID and status
API --> Web UI: 201 Created / allocation summary
```

Flow B: Expiry Alert Generation

```
Alert Scheduler -> Database: Find licenses/warranties nearing expiry
Database --> Alert Scheduler: Matching records
Alert Scheduler -> Alert Service: Evaluate threshold and existing alert
Alert Service -> Database: Create alert if not already present
Alert Service -> Notification Adapter: Send optional email
Notification Adapter --> Alert Service: Delivery status
Alert Service -> Database: Store delivery result
Web UI -> API: GET /api/alerts
```

```
API -> Database: Fetch alerts visible to current role
Database --> API: Alert list
API --> Web UI: Alert centre results
```

4.3 API Design

Endpoint	Method	Purpose / Request	Success Response	Common Errors
/api/assets	GET	List/search assets; query filters such as status, category, department	200: paginated asset list	401 unauthenticated; 403 forbidden
/api/assets	POST	Create asset; JSON includes name, category, assetTag, status, location	201: assetId and asset summary	400 validation error; 409 duplicate tag
/api/allocations	POST	Allocate asset; JSON includes assetId, assigneeId, allocationDate	201: allocationId, status	403 forbidden; 409 unavailable asset
/api/assets/{id}/return	POST	Return an allocated asset; optional return date and notes	200: updated asset status	404 not found; 409 not allocated
/api/licenses	POST	Create license record; product, entitlement, expiryDate, masked reference	201: licenseId	400 invalid fields; 403 forbidden
/api/alerts	GET	List alerts; optional status and type filters	200: alert list	401 unauthenticated; 403 forbidden
/api/reports/expiry	GET	Report licenses/warranties expiring within a supplied number of days	200: report rows	400 invalid range; 403 forbidden

API design notes: use JSON; validate all inputs on the server; return consistent error objects such as {"errorCode":"VALIDATION_ERROR","message":"Check required fields","fieldErrors":{...}}. Do not return stack traces or secrets. Apply pagination to list endpoints.

4.4 Error Handling, Logging and Monitoring

- Return appropriate HTTP status codes and clear, non-sensitive messages.
- Log request ID, actor ID where available, endpoint, outcome and timestamp; never log passwords, session tokens or full license secrets.
- Monitor API error rate, response time, failed logins, alert-generation failures, notification delivery failures and database connectivity.
- Show validation errors to users and retain failed CSV import row details for correction.
- Use transactions for allocation/return and related audit writes; handle conflicts without partial updates.

4.5 UX Design

Prioritize dashboard cards for total assets, assigned assets, available assets, expiring licenses/warranties and overdue maintenance. Use consistent status labels and filters. Forms should indicate required fields, dates and validation errors. Provide a confirmation step for retirement and reassignment. Keep auditor views read-only and show alert severity, due date, owner and resolution status.

4.6 Open Issues and Next Steps

- Confirm final implementation framework and database.
- Confirm organizational expiry thresholds (for example 30/14/7 days) and maintenance escalation rules.
- Decide whether email notification and PDF export are included in the first release.
- Create visual UML diagrams from the supplied logical/sequence specifications if required by the evaluator.
- Implement backup/restore and access-control tests before any production use.

5. Appendices

5.1 Glossary

Asset lifecycle: progression from registration through assignment, maintenance and retirement. Entitlement: authorized license quantity or usage. Idempotent alert: repeated scheduler runs do not create duplicate alerts for the same event. RBAC: Role-Based Access Control.

5.2 References

- IEEE 42010, Systems and software engineering — Architecture description.
- OWASP Application Security Verification Standard (ASVS) and OWASP Top 10, as applicable.
- NIST SP 800-160, Systems Security Engineering, as applicable.
- Project Software Requirements Specification (ITAMS SRS v1.0).

5.3 Tools

PlantUML or draw.io for diagrams; Postman for API checks; Git for version control; relational database administration tools; automated unit/integration testing framework.